

JDBC & Repository Pattern

Come leggere e scrivere su MySQL in Java con architettura pulita, usando JDBC puro e il pattern Repository.

INDICE

1. Cos'è JDBC
2. Architettura
3. Setup MySQL
4. Classe Modello
5. ConnectionManager
6. Repository
7. Main / Utilizzo

1 Cos'è JDBC

JDBC (Java Database Connectivity) è l'API standard di Java per comunicare con database relazionali. Funziona come uno strato di astrazione: il tuo codice parla con l'API JDBC, e il *driver specifico* (nel nostro caso `mysql-connector-j`) traduce le chiamate nel protocollo nativo del database.

💡 **Analogia utile:** JDBC è come una presa elettrica universale. Il tuo dispositivo (codice Java) si collega sempre allo stesso standard; è il trasformatore (driver) che si adatta al paese (database).

Il flusso di una chiamata JDBC è sempre lo stesso:

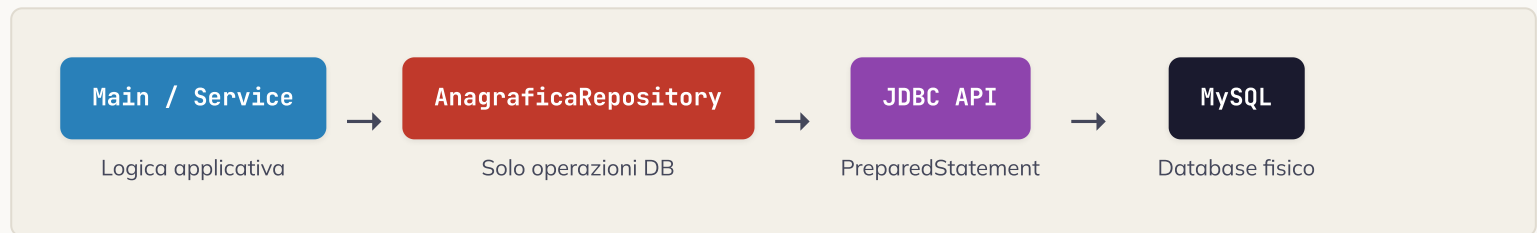
#	PASSO	CLASSE/METODO CHIAVE
1	Aprire una connessione al DB	<code>DriverManager.getConnection()</code>

#	PASSO	CLASSE/METODO CHIAVE
2	Creare uno statement con la query	<code>conn.prepareStatement(sql)</code>
3	Legare i parametri (bind)	<code>ps.setString(1, valore)</code>
4	Eseguire la query	<code>ps.executeQuery()</code> / <code>ps.executeUpdate()</code>
5	Leggere il ResultSet (solo SELECT)	<code>rs.getString("colonna")</code>
6	Chiudere tutte le risorse	<code>try-with-resources</code>

⚠ Mai usare **Statement** puro! Usa sempre **PreparedStatement**: previene le SQL Injection e migliora le performance grazie alla cache del piano di esecuzione.

2 Architettura del progetto

Invece di sparpagliare query SQL ovunque nel codice, usiamo il **Repository Pattern**: un singolo componente centralizza tutta la logica di accesso ai dati per una entità. Il resto del codice non sa nulla di SQL o JDBC.



La struttura dei file del progetto è la seguente:

```
src/
├─ java/com/example/jdbc/
│   ├─ model/
│   │   └─ Anagrafica.java           // POJO entità
│   └─ repository/
│       ├─ AnagraficaRepository.java // interfaccia
│       └─ AnagraficaRepositoryImpl.java
│   └─ db/
│       ├─ ConnectionManager.java    // URL, user e password hardcoded
│       └─ Main.java
└─ lib/
    └─ mysql-connector-j-9.2.0.jar    // driver MySQL (vedi sezione 3)
```

3

Setup: MySQL e librerie necessarie

Schema della tabella

 **tabella: anagrafica**

COLONNA	TIPO	VINCOLI
id	INT	PK • AUTO_INCREMENT
nome	VARCHAR(100)	NOT NULL
cognome	VARCHAR(100)	NOT NULL

COLONNA	TIPO	VINCOLI
anno_nascita	INT	NOT NULL

```

schema.sql

CREATE DATABASE IF NOT EXISTS anagrafica_db;
USE anagrafica_db;

CREATE TABLE IF NOT EXISTS anagrafica (
    id            INT            AUTO_INCREMENT PRIMARY KEY,
    nome          VARCHAR(100) NOT NULL,
    cognome       VARCHAR(100) NOT NULL,
    anno_nascita  INT            NOT NULL
);

```

Libreria richiesta: MySQL Connector/J

L'unica libreria esterna necessaria è il driver JDBC ufficiale di MySQL, distribuito come singolo file `.jar`. JDBC stesso (`java.sql.*`) è già incluso nel JDK standard, non richiede download.

LIBRERIA	VERSIONE	DIMENSIONE	DOWNLOAD
mysql-connector-j	9.2.0	~2.5 MB	dev.mysql.com/downloads/connector/j/

💡 Quale file scaricare? Nella pagina di download seleziona *"Platform Independent"* e scarica lo ZIP/TAR. Al suo interno troverai il file `mysql-connector-j-9.2.0.jar`. Copialo nella cartella `lib/` del tuo progetto.

Come aggiungere il JAR al classpath

A seconda dell'ambiente di sviluppo che usi, il modo per includere il JAR cambia:

► NetBeans

Apache NetBeans

IDE

1. Nella finestra "Projects", espandi il tuo progetto
2. Tasto destro su "Libraries" → "Add JAR/Folder..."
3. Naviga e seleziona `mysql-connector-j-9.2.0.jar`
4. Clicca "Open" – il JAR appare sotto "Libraries"

In alternativa, tramite proprietà del progetto:

1. Tasto destro sul progetto → "Properties"
2. Categoria "Libraries" → scheda "Compile" → "Add JAR/Folder..."
3. Seleziona il file e conferma con "OK"

► Visual Studio Code

VS Code (con Extension Pack for Java)

EDITOR

Prerequisito: installa l'estensione "Extension Pack for Java" (include Language Support for Java, Debugger for Java, ecc.)

Metodo 1 – tramite pannello JAVA PROJECTS (consigliato):

1. Apri la vista "Java Projects" nella barra laterale
2. Espandi il tuo progetto → sezione "Referenced Libraries"
3. Clicca il "+" accanto a "Referenced Libraries"

4. Seleziona `mysql-connector-j-9.2.0.jar`

Metodo 2 – tramite `settings.json` del workspace:

Aggiungi questa riga nel file `.vscode/settings.json`:

```
{
  "java.project.referencedLibraries": [
    "lib/mysql-connector-j-9.2.0.jar"
  ]
}
```

💡 **Suggerimento per VS Code:** posiziona il JAR nella cartella `lib/` alla radice del progetto prima di aggiungerlo. In questo modo il percorso relativo nel `settings.json` funzionerà correttamente su qualsiasi macchina che clona il progetto.

La stringa di connessione JDBC

La stringa di connessione è il parametro più critico: contiene host, porta, nome del database e opzioni aggiuntive. Per MySQL la forma generale è:

formato JDBC URL per MySQL

URL

```
jdbc:mysql://<host>:<porta>/<nome_database>?<parametri>
```

Esempio locale:

```
jdbc:mysql://localhost:3306/anagrafica_db?useSSL=false&serverTimezone=UTC
```

Parametri utili più comuni:

<code>useSSL=false</code>	→ disabilita SSL (sviluppo locale)
<code>serverTimezone=UTC</code>	→ evita errori di fuso orario

```
autoReconnect=true    → riconnessione automatica se cade il link
characterEncoding=UTF-8 → gestione corretta dei caratteri
```

🌐 Riferimento online: il sito connectionstrings.com/mysql raccoglie la sintassi delle stringhe di connessione per tutti i principali database e driver, con esempi pronti all'uso per MySQL Connector/J, ODBC, .NET e molti altri. È utile anche per scenari avanzati come connessioni SSL, socket Unix, o MySQL su Azure/AWS.

4 Classe Modello (POJO)

La classe `Anagrafica` rappresenta una singola riga della tabella. Non contiene alcuna logica di accesso al DB: è un semplice contenitore di dati (*Plain Old Java Object*).

model/Anagrafica.java

MODEL

```
package com.example.jdbc.model;

public class Anagrafica {

    private Integer id;           // null finché non è salvata nel DB
    private String  nome;
    private String  cognome;
    private int     annoNascita;

    // Costruttore per INSERT (senza id)
    public Anagrafica(String nome, String cognome, int annoNascita) {
        this.nome      = nome;
    }
}
```

```

        this.cognome      = cognome;
        this.annoNascita = annoNascita;
    }

    // Costruttore per lettura dal DB (con id)
    public Anagrafica(int id, String nome, String cognome, int annoNascita) {
        this.id          = id;
        this.nome         = nome;
        this.cognome      = cognome;
        this.annoNascita = annoNascita;
    }

    // Getter e Setter
    public Integer getId()          { return id; }
    public void    setId(int id)    { this.id = id; }

    public String  getNome()        { return nome; }
    public void    setName(String v) { this.nome = v; }

    public String  getCognome()      { return cognome; }
    public void    setCognome(String v) { this.cognome = v; }

    public int     getAnnoNascita()   { return annoNascita; }
    public void    setAnnoNascita(int v) { this.annoNascita = v; }

    @Override
    public String toString() {
        return "Anagrafica{id=" + id +
            ", nome='" + nome + '\'' +
            ", cognome='" + cognome + '\'' +
            ", annoNascita=" + annoNascita + '}';
    }

```



```
}  
}
```

5 ConnectionManager

Un componente centralizzato che espone un unico metodo statico `getConnection()`. I parametri di connessione sono definiti come costanti direttamente nel codice: semplice e immediato per un progetto didattico o prototipo.

db/ConnectionManager.java

CONFIG

```
package com.example.jdbc.db;  
  
import java.sql.Connection;  
import java.sql.DriverManager;  
  
public class ConnectionManager {  
  
    // — Parametri di connessione — modifica questi valori —  
    private static final String URL =  
        "jdbc:mysql://localhost:3306/anagrafica_db?useSSL=false&serverTimezone=UTC";  
    private static final String USER = "root";  
    private static final String PASSWORD = "la_tua_password";  
  
    /**  
     * Restituisce una nuova connessione al database.  
     * Il chiamante è responsabile di chiuderla (preferibilmente con try-with-resources).  
     */  
}
```

```
public static Connection getConnection() throws Exception {  
    return DriverManager.getConnection(URL, USER, PASSWORD);  
}  
  
// Classe di sola utilità: costruttore privato  
private ConnectionManager() {}  
}
```

⚠ Solo per uso didattico! Hardcodare le credenziali nel sorgente va bene per imparare o per prototipi locali, ma non farlo mai in produzione: le password nel codice finiscono nei sistemi di versioning (Git) e sono visibili a chiunque abbia accesso al repository. In un progetto reale usa variabili d'ambiente o un file di configurazione esterno non versionato.

✅ **Nota:** In un'applicazione reale aggiungeresti HikariCP come pool di connessioni direttamente in questa classe, sostituendo `DriverManager.getConnection()` con `hikariDataSource.getConnection()`. L'interfaccia verso il Repository rimane identica.

6 Repository: interfaccia e implementazione

Prima definiamo il contratto (interfaccia), poi l'implementazione concreta con JDBC. Questo disaccoppiamento permette di sostituire l'implementazione (es. con un ORM) senza toccare il codice che usa il repository.

Interfaccia

repository/AnagraficaRepository.java

INTERFACE

```
package com.example.jdbc.repository;

import com.example.jdbc.model.Anagrafica;
import java.util.List;
import java.util.Optional;

public interface AnagraficaRepository {

    /** Inserisce un nuovo record e imposta l'id generato sull'oggetto */
    void save(Anagrafica a) throws Exception;

    /** Legge tutti i record */
    List<Anagrafica> findAll() throws Exception;

    /** Cerca per id; ritorna Optional.empty() se non trovato */
    Optional<Anagrafica> findById(int id) throws Exception;

    /** Aggiorna nome, cognome e anno di un record esistente */
    void update(Anagrafica a) throws Exception;

    /** Cancella il record con l'id specificato */
    void deleteById(int id) throws Exception;
}
```

Implementazione JDBC

repository/AnagraficaRepositoryImpl.java

WRITE

```

package com.example.jdbc.repository;

import com.example.jdbc.db.ConnectionManager;
import com.example.jdbc.model.Anagrafica;

import java.sql.*;
import java.util.*;

public class AnagraficaRepositoryImpl implements AnagraficaRepository {

    // — Query SQL come costanti —————
    private static final String INSERT =
        "INSERT INTO anagrafica (nome, cognome, anno_nascita) VALUES (?, ?, ?)";
    private static final String SELECT_ALL =
        "SELECT id, nome, cognome, anno_nascita FROM anagrafica ORDER BY cognome, nome";
    private static final String SELECT_BY_ID =
        "SELECT id, nome, cognome, anno_nascita FROM anagrafica WHERE id = ?";
    private static final String UPDATE =
        "UPDATE anagrafica SET nome = ?, cognome = ?, anno_nascita = ? WHERE id = ?";
    private static final String DELETE =
        "DELETE FROM anagrafica WHERE id = ?";

    // — SAVE (INSERT) —————
    @Override
    public void save(Anagrafica a) throws Exception {
        // try-with-resources chiude automaticamente Connection e Statement
        try (Connection conn = ConnectionManager.getConnection();
            PreparedStatement ps = conn.prepareStatement(INSERT,
                Statement.RETURN_GENERATED_KEYS)) {

            ps.setString(1, a.getNome());
            ps.setString(2, a.getCognome());

```

```

        ps.setInt(3, a.getAnnoNascita());
        ps.executeUpdate();

        // Recuperiamo l'id auto-generato dal DB
        try (ResultSet keys = ps.getGeneratedKeys()) {
            if (keys.next()) {
                a.setId(keys.getInt(1));
            }
        }
    }
}

// — FIND ALL (SELECT) —————
@Override
public List<Anagrafica> findAll() throws Exception {
    List<Anagrafica> result = new ArrayList<>();

    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(SELECT_ALL);
        ResultSet rs = ps.executeQuery()) {

        while (rs.next()) {
            result.add(mapRow(rs)); // metodo di mapping riusabile
        }
    }
    return result;
}

// — FIND BY ID —————
@Override
public Optional<Anagrafica> findById(int id) throws Exception {
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(SELECT_BY_ID)) {

```

```

        ps.setInt(1, id);

        try (ResultSet rs = ps.executeQuery()) {
            return rs.next()
                ? Optional.of(mapRow(rs))
                : Optional.empty();
        }
    }
}

// — UPDATE —————
@Override
public void update(Anagrafica a) throws Exception {
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(UPDATE)) {

        ps.setString(1, a.getNome());
        ps.setString(2, a.getCognome());
        ps.setInt(3, a.getAnnoNascita());
        ps.setInt(4, a.getId());
        ps.executeUpdate();
    }
}

// — DELETE —————
@Override
public void deleteById(int id) throws Exception {
    try (Connection conn = ConnectionManager.getConnection();
        PreparedStatement ps = conn.prepareStatement(DELETE)) {

        ps.setInt(1, id);
        ps.executeUpdate();
    }
}

```

```

    }
}

// — HELPER: mappa una riga del ResultSet in un oggetto —————
private Anagrafica mapRow(ResultSet rs) throws SQLException {
    return new Anagrafica(
        rs.getInt    ("id"),
        rs.getString("nome"),
        rs.getString("cognome"),
        rs.getInt    ("anno_nascita")
    );
}
}

```

💡 try-with-resources: da Java 7 in poi, qualunque oggetto che implementa `AutoCloseable` (come `Connection`, `PreparedStatement`, `ResultSet`) viene chiuso automaticamente anche in caso di eccezione. È il modo corretto di gestire le risorse JDBC.

7 Main: esempio di utilizzo completo

Il `Main` usa esclusivamente l'interfaccia `AnagraficaRepository`. Non c'è traccia di SQL o JDBC: quella complessità è nascosta nel repository.

Main.java

READ + WRITE

```
package com.example.jdbc;

import com.example.jdbc.model.Anagrafica;
import com.example.jdbc.repository.*;

import java.util.List;

public class Main {

    public static void main(String[] args) throws Exception {

        AnagraficaRepository repo = new AnagraficaRepositoryImpl();

        // — 1. INSERT —————
        Anagrafica mario = new Anagrafica("Mario", "Rossi", 1985);
        Anagrafica laura = new Anagrafica("Laura", "Bianchi", 1992);
        Anagrafica luca = new Anagrafica("Luca", "Verdi", 1978);

        repo.save(mario);    // mario.getId() ora ha il valore reale del DB
        repo.save(laura);
        repo.save(luca);

        System.out.println("Inseriti: " + mario + ", " + laura + ", " + luca);

        // — 2. SELECT ALL —————
        List<Anagrafica> tutti = repo.findAll();
        System.out.println("\n— Tutti i record —");
        tutti.forEach(System.out::println);

        // — 3. SELECT BY ID —————
        repo.findById(mario.getId())
            .ifPresentOrElse(
```



```

        a → System.out.println("\nTrovato per id: " + a),
        () → System.out.println("Non trovato")
    );

    // — 4. UPDATE —————
    mario.setNome("Mario Angelo");
    mario.setAnnoNascita(1986);
    repo.update(mario);
    System.out.println("\nAggiornato: " + mario);

    // — 5. DELETE —————
    repo.deleteById(luca.getId());
    System.out.println("\nEliminato id=" + luca.getId());

    // — 6. Verifica finale —————
    System.out.println("\n— Record finali —");
    repo.findAll().forEach(System.out::println);
}
}

```

Output atteso

console output

```

Inseriti: Anagrafica{id=1, nome='Mario', cognome='Rossi', annoNascita=1985}
         Anagrafica{id=2, nome='Laura', cognome='Bianchi', annoNascita=1992}
         Anagrafica{id=3, nome='Luca', cognome='Verdi', annoNascita=1978}

— Tutti i record —
Anagrafica{id=2, nome='Laura', cognome='Bianchi', annoNascita=1992}

```

```
Anagrafica{id=1, nome='Mario', cognome='Rossi', annoNascita=1985}
Anagrafica{id=3, nome='Luca', cognome='Verdi', annoNascita=1978}

Trovato per id: Anagrafica{id=1, nome='Mario', cognome='Rossi', annoNascita=1985}

Aggiornato: Anagrafica{id=1, nome='Mario Angelo', cognome='Rossi', annoNascita=1986}

Eliminato id=3

— Record finali —
Anagrafica{id=2, nome='Laura', cognome='Bianchi', annoNascita=1992}
Anagrafica{id=1, nome='Mario Angelo', cognome='Rossi', annoNascita=1986}
```

Riepilogo: pro e contro di questo approccio

ASPETTO	VALUTAZIONE	NOTE
Controllo sull'SQL	✓ Totale	Ogni query è esplicita e ottimizzabile
Sicurezza (SQL Injection)	✓ Protetto	PreparedStatement con bind dei parametri
Boilerplate	✗ Alto	Ogni operazione richiede apertura/chiusura risorse
Leggibilità del codice	✓ Buona	Il pattern Repository isola la complessità
Performance	✓ Massima	Nessun overhead ORM; da combinare con HikariCP
Scalabilità del progetto	✗ Media	Molte entità → molto codice ripetitivo
Curva di apprendimento	✓ Bassa	Non richiede conoscenza di framework esterni

Tutorial JDBC + Repository Pattern — Java & MySQL · Riferimento: [java.sql API Docs](#) · [MySQL Connector/J](#)